Name : Palak Singh Roll no : 202403123 Class : TYCS-B

Practical No. 8

Object Identification and Counting on a Web Page

Tool: Selenium WebDriver with Java
Platform: GitHub Codespaces

1. Aim

To identify and count different types of objects present on a web page using Selenium WebDriver, including:

- Links
- Images
- Buttons
- Input fields

The Selenium-generated count will then be compared with the student's manual count of the webpage.

2. Requirements

The following are required:

1. GitHub account
2. GitHub Codespaces
3. Java
4. Maven
5. Chromium browser
6. ChromeDriver
7. Selenium WebDriver
8. JUnit

Note: Students do not need to install an IDE or any software on their personal computer. The practical is performed inside GitHub Codespaces.

3. Open the Practical Repository

Step 1

Log in to your GitHub account.

Step 2

Open the repository link provided by your teacher.

https://github.com/Conceicaodsouza/STQA-Practical-8-ObjectCounting

Step 3

Click:

Code → Codespaces

Step 4

Click:

Create codespace on main

Wait for the Codespace to open completely.

You should eventually see a terminal similar to:

@StudentName ➜ /workspaces/STQA-Practical-8-ObjectCounting (main) $

4. Check the Project Files

In the terminal, type:

ls

You should see:

README.md

pom.xml

src

students.html

The important files are:

File/Folder	Purpose
students.html	Webpage that will be tested
pom.xml	Maven configuration
Src	Contains the Selenium test program

5. Check Java

In the terminal, type:

java -version

A Java version should be displayed.

For example:

openjdk version "21..."

If Java is already available, continue to the next step.

6. Check Maven

Type:

mvn -version

If Maven is installed

You will see information similar to:

Apache Maven ...

Java version: ...

Continue to Step 7.

If you get:

mvn: command not found

install Maven:

sudo apt-get update

Then:

sudo apt-get install -y maven

Check again:

mvn -version

7. Check Chromium

Type:

which chromium

Expected result:

/usr/bin/chromium

Then type:

chromium --version

You should see the installed Chromium version.

If it is not available run –

sudo apt-get update and then

sudo apt-get install -y chromium chromium-driver

Now, check chromium version

8. Check ChromeDriver

Type:

which chromedriver

Expected result:

/usr/bin/chromedriver

Then:

chromedriver --version

You should see the ChromeDriver version.

Important

The Chromium and ChromeDriver versions should have the **same major version number**.

For example:

Chromium 151.x.x

ChromeDriver 151.x.x

9. Open the Webpage

The webpage that will be tested is:

students.html

It is located in the root folder of the project.

Step 1

In the Explorer panel on the left side of VS Code, find:

students.html

Step 2

Right-click on:

students.html

Step 3

Select:

Open with Live Server

The webpage will open in the browser/preview available in the Codespace.

10. Observe the Webpage

Now carefully examine the webpage.

The webpage contains different HTML objects.

You need to manually identify and count:

A. Links

Count all hyperlinks present on the webpage.

B. Images

Count all images displayed on the webpage.

C. Buttons

Count all buttons.

D. Input Fields

Count all input elements such as:

- Text boxes
- Other <input> elements

11. Record Your Manual Count

Before running Selenium, record your observations.

Object Type	**Manual Count**
Links	_____
Images	_____
Buttons	_____
Input Fields	_____
Total Objects	_____

Calculate:

Total Objects =

Links + Images + Buttons + Input Fields

12. Open the Selenium Program

In the Explorer panel, navigate to:

src

└── test

└── java

└── practical

└── ObjectCountingTest.java

Open:

ObjectCountingTest.java

13. Understand the Program

The program creates a Selenium WebDriver object and opens the webpage.

The important statements are:

Count Links

int linkCount = driver.findElements(By.tagName("a")).size();

The <a> HTML tag represents hyperlinks.

Count Images

int imageCount = driver.findElements(By.tagName("img")).size();

The <img> HTML tag represents images.

Count Buttons

int buttonCount = driver.findElements(By.tagName("button")).size();

The <button> HTML tag represents buttons.

Count Input Fields

int inputCount = driver.findElements(By.tagName("input")).size();

The <input> HTML tag represents input elements.

Calculate Total Objects

int totalObjects = linkCount + imageCount + buttonCount + inputCount;

14. Run the Selenium Program

Return to the terminal.

Make sure you are in:

/workspaces/STQA-Practical-8-ObjectCounting

Then execute:

mvn clean test

Press **Enter**.

Wait for Maven and Selenium to complete execution.

15. Observe the Selenium Output

The program will display an output similar to:

```
==========================================
    WEB PAGE OBJECT ANALYSIS
==========================================


Total Links          3
Total Images         2
Total Buttons        2
```

Total Input Fields : 4

- -

Total Objects 11

==========

The numbers shown above are only an example. Students must use the actual numbers displayed by their program.

16. Verify BUILD SUCCESS

At the end of the Maven output, look for:

BUILD SUCCESS

This indicates that the Selenium test executed successfully.

17. Compare Manual and Automated Counts

Now compare your manual count with the Selenium count.

Object Type	**Manual Count**	**Selenium Count**
Links	_____	_____
Images	_____	_____
Buttons	_____	_____
Input Fields	_____	_____
Total Objects	_____	_____

The two counts should match.

18. Result

Write the following in your practical record:

Result: The webpage objects were successfully identified and counted using Selenium WebDriver. The number of links, images, buttons, and input fields was obtained and compared with the manual count.

19. Important Commands

Students can refer to this section if they forget a command.

Display project files

ls

Check Java

java -version

Check Maven

mvn -version

Check Chromium

which chromium

chromium --version

Check ChromeDriver

which chromedriver

chromedriver --version

Install Maven if required

sudo apt-get update

sudo apt-get install -y maven

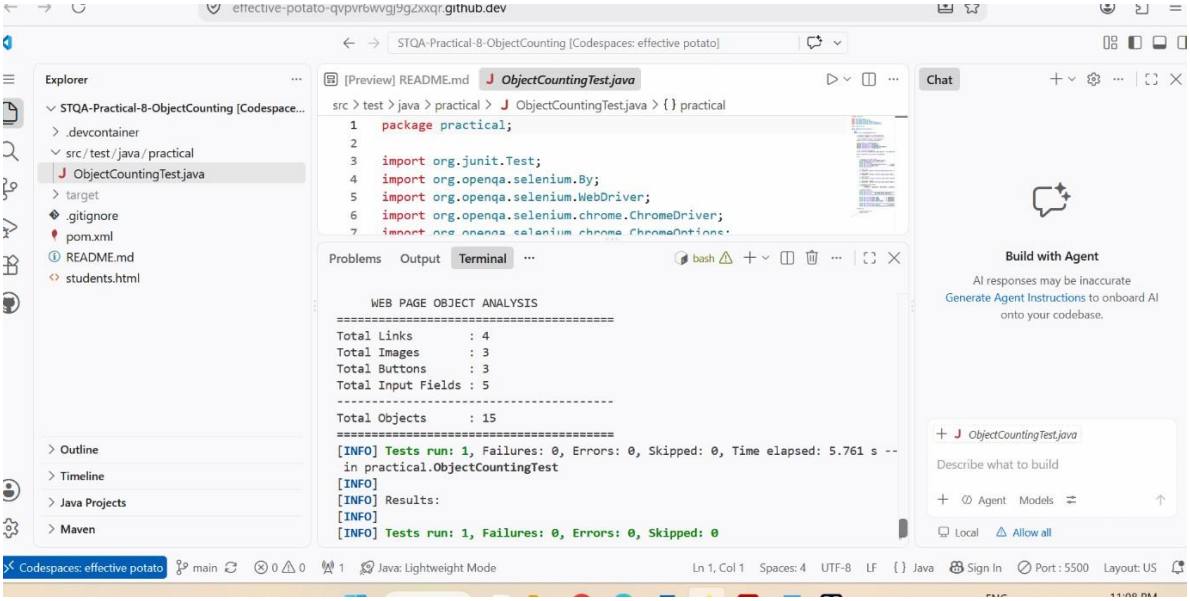
Install Chromium and ChromeDriver if required

sudo apt-get update

sudo apt-get install -y chromium chromium-driver

Execute the Selenium practical

mvn clean test



Name : Palak Singh Roll no : 202403123 Class : TYCS-B

Practical 9 – List and Combo Box Item Verification

Tool: Selenium WebDriver
Language: Java
Build Tool: Maven
Platform: GitHub Codespaces

1. Open the Repository

1. Log in to GitHub using your student account.
2. Open the Practical 9 repository provided by the teacher - **https://github.com/consy16/STQA-Practical-9-List-ComboBox-New**
3. Open the repository in **GitHub Codespaces**.
4. Click Code >> Codespaces >> Create codespace on new. Wait until the Codespace is fully loaded.
5. Open the **Terminal**.

---

2. Check Java

In the terminal, type:

java -version

Java 21 should be displayed.

---

3. Check Maven

Type:

mvn -version

Maven version and Java version should be displayed.

---

4. Check Project Files

Type:

find src -type f

The following files should be present:

src/test/resources/list-combobox.html
src/test/java/practical/ListComboBoxTest.java

---

5. Check Google Chrome

Type:

google-chrome --version

Chrome version should be displayed.

---

6. Check ChromeDriver

Type:

chromedriver --version

The ChromeDriver version should match the installed Chrome version.

---

7. Start the Local Web Server

The Python server may not be available in the Codespace, so use Java's built-in web server.

Type:

jwebserver -p 8000 -d /workspaces/STQA-Practical-9-List-ComboBox-New/src/test/resources

You should see:

Serving ... on 127.0.0.1 port 8000
URL http://127.0.0.1:8000/

Keep this terminal running.

---

8. Forward Port 8000

1. Open the **PORTS** tab in Codespaces.
2. Click **Forward a port**.
3. Enter:

8000

4. Press **Enter**.
5. Open the forwarded port in the browser.

The **List and Combo Box** webpage should be displayed.

---

9. Run the Selenium Test

1. Keep the web-server terminal running.
2. Open a **new terminal**:

Terminal → New Terminal

3. Make sure you are inside the project folder:

/workspaces/STQA-Practical-9-List-ComboBox-New

4. Run:

mvn test

5. Wait for the test to finish.

---

10. Verify the Result

The final output should show:

BUILD SUCCESS

This confirms that the Selenium test was executed successfully.

---

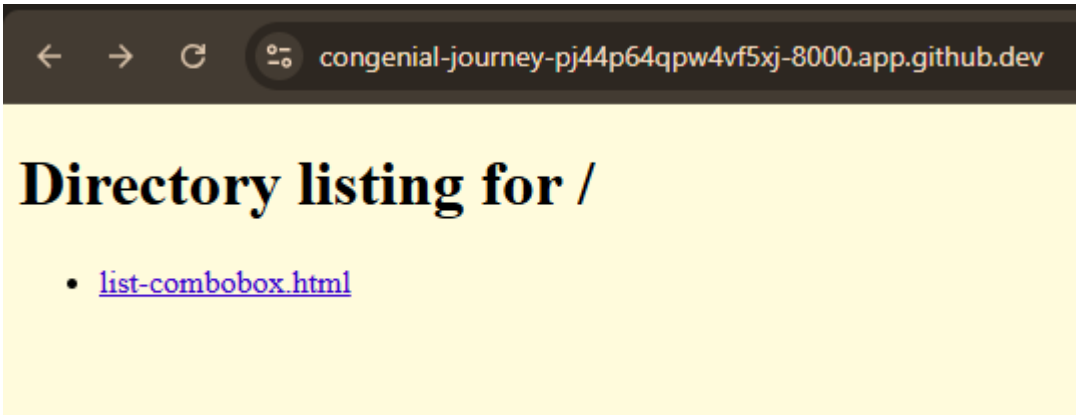
11. Practical Verification

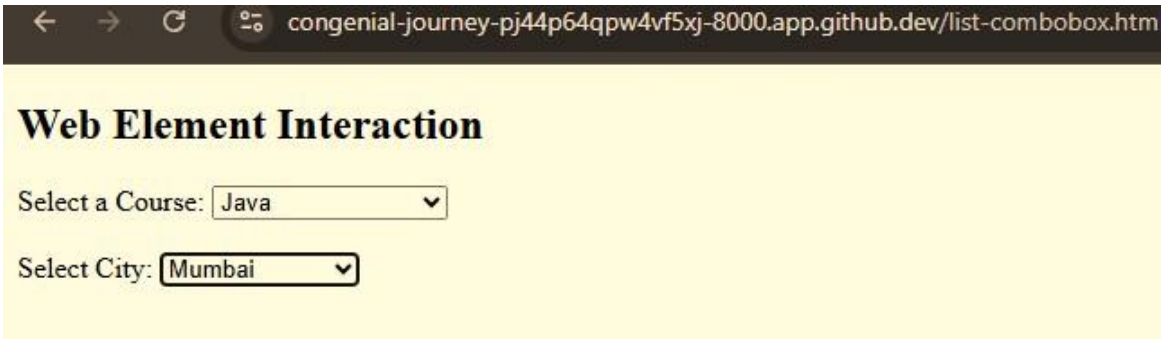
The Selenium program verifies:

- List/Combo Box identification
- Number of items
- Default selected value
- Selection of items from the list/combo box

Name : Palak Singh Roll no : 202403123 Class : TYCS-B

Output:






```
================================
LIST / COMBOBOX PRACTICAL
================================
ComboBox found successfully.
Option selected: Selenium
Test completed successfully.
================================
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.200 s -- in practical.ListComboBoxTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  4.619 s
[INFO] Finished at: 2026-09-30T03:05:16Z
[INFO] ------------------------------------------------------------------------
@Dinku-19 ➜/workspaces/STQA-Practical-9-List-ComboBox-New (main) $
```

Name : Palak Singh Roll no : 202403123 Class : TYCS-B

Practical 10 – Load Testing Using Apache JMeter

Title: Load Testing Using Apache JMeter
Tool: Apache JMeter
Objective: To design and execute load testing scenarios for a web application and analyze response time, throughput, and error percentage.

---

1. Install Java

1. Open the official Java website:

   https://adoptium.net/temurin/releases/?version=21

   Download - Temurin 21 → Windows → x64 → JDK → MSI

2. Download the msi file. It will look like - OpenJDK21U-jdk_x64_windows_hotspot_21.... msi
3. Double click the downloaded .msi file and complete the installation
4. Open **Command Prompt** and verify Java:

   java -version

5. You should see - openjdk version "21.x.x"
6. Then type - javac -version
7. You should see - javac 21.x.x

---

2. Download Apache JMeter

1. Open the official Apache JMeter download page:
   Apache JMeter - Download Apache JMeter
2. Download the **Binary ZIP file** of Apache JMeter. You should see - apache-jmeter-5.6.3.zip
3. Extract the ZIP file to C:\JMeter.
4. Open the extracted JMeter folder.
5. Open the folder – bin
6. Double-click:

jmeter.bat

A black Command Prompt window may appear first. **Do not close it.**

After a few seconds, the **Apache JMeter GUI** should open.

---

3. Create a Folder for the Practical

1. Open **Command Prompt**.
2. Type:

```
mkdir C:\STQA-JMeter
```

3. Move to the folder:

```
cd C:\STQA-JMeter
```

---

4. Create the Web Page

1. Open **Notepad**.
2. Enter the following HTML:

```
<!DOCTYPE html>
<html>
<head>
   <title>STQA Load Testing</title>
</head>
<body>
   <h1>STQA Load Testing Application</h1>
   <p>This page is used for Apache JMeter load testing.</p>
   <button>Submit</button>
</body>
</html>
```

3. Save the file as:

```
index.html
```

In the Save As window, set **Save as type** to: all files

4. Save it inside:

```
C:\STQA-JMeter
```

---

5. Start the Local Web Server

1. Open Command Prompt.
2. Go to:

cd C:\STQA-JMeter

3. Then run -

   java -m jdk.httpserver

   You should see - Binding to loopback address 127.0.0.1, port 8000

4. Test the web page

Open Chrome/Edge and enter - http://localhost:8000

6. Verify that the **STQA Load Testing Application** page is displayed.
7. **Keep the Command Prompt open** while performing the JMeter test.

---

6. Create the JMeter Test Plan

1. Open **Apache JMeter**.
2. Right-click **Test Plan**.
3. Select:

Add → Threads (Users) → Thread Group

---

7. Configure Thread Group – Scenario 1

Select **Thread Group** and enter:

Number of Threads (users): 10
Ramp-up period (seconds): 10
Loop Count: 5

Therefore:

10 users × 5 loops = 50 requests

---

8. Add HTTP Request

1. Right-click **Thread Group**.
2. Select:

Add → Sampler → HTTP Request

3. Enter:

Name: Open STQA Web Page
Protocol: http
Server Name or IP: localhost
Port Number: 8000
Method: GET
Path: /

---

9. Add Listeners

View Results Tree

Right-click **Thread Group**:

Add → Listener → View Results Tree

Summary Report

Right-click **Thread Group**:

Add → Listener → Summary Report

Graph Results

Right-click **Thread Group**:

Add → Listener → Graph Results

The final structure should be:

Test Plan
└── Thread Group
  ├── HTTP Request
  ├── View Results Tree
  ├── Summary Report
  └── Graph Results

---

10. Save the Test Plan

Select:

File → Save Test Plan As

Save as:

STQA_Load_Test.jmx

inside:

C:\STQA-JMeter

---

11. Execute Scenario 1

1. Select:

Run → Clear All

2. Click **Summary Report**.
3. Click the green **Start** ▶ button.
4. Wait for the test to finish.
5. Check **Summary Report**.

Expected:

Samples = 50
Error % = 0.00%

Record:

- Samples
- Average Response Time
- Minimum Response Time
- Maximum Response Time
- Error %
- Throughput

6. Open **View Results Tree** and verify that the requests are successful.
7. Open **Graph Results** to view the response-time graph.

---

12. Execute Scenario 2 – Higher Load

Change the Thread Group settings to:

Number of Threads (users): 50
Ramp-up period (seconds): 20
Loop Count: 10

Therefore:

50 users × 10 loops = 500 requests

Run the Test

1. Select:

Run → Clear All

2. Select **Summary Report**.
3. Click **Start** ▶️.
4. Wait for the test to finish.
5. Record:

- Samples
- Average Response Time
- Minimum Response Time
- Maximum Response Time
- Error %
- Throughput

6. Open **Graph Results** and observe the graph.

---

13. Result Analysis

Compare the results of both scenarios.

Parameter	Scenario 1	Scenario 2
Users	10	50
Loop Count	5	10
Total Requests	50	500
Average Response Time	_____	_____
Maximum Response Time	_____	_____
Throughput	_____	_____
Error %	_____	_____

Interpretation

- **Response Time:** Time taken by the web server to respond to a request.
- **Throughput:** Number of requests processed per unit of time.
- **Error %:** Percentage of requests that failed.

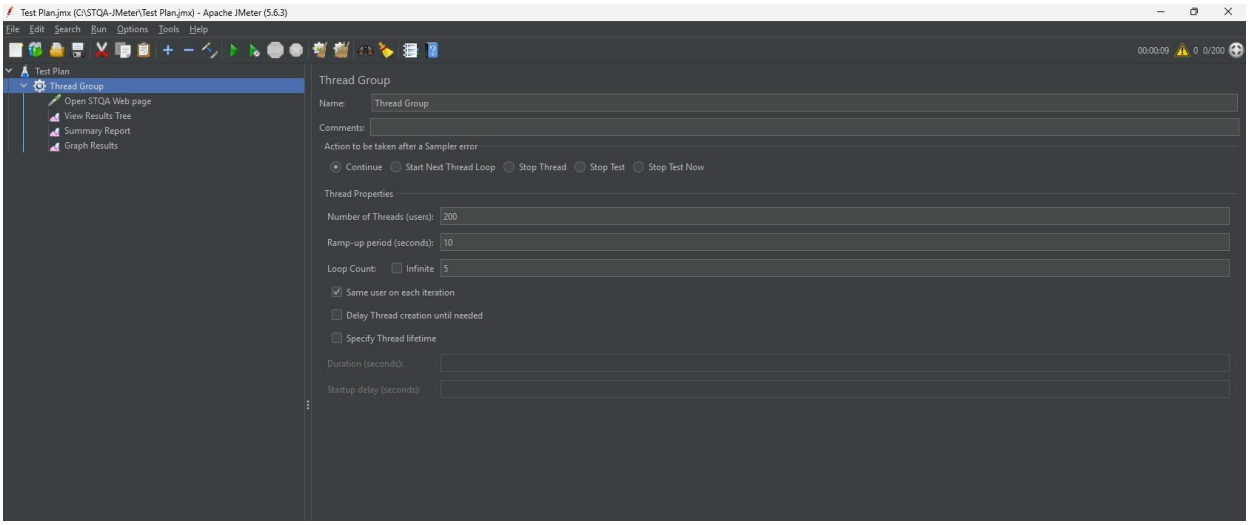
- Increasing the number of users and requests allows the behavior of the application under a higher load to be observed.

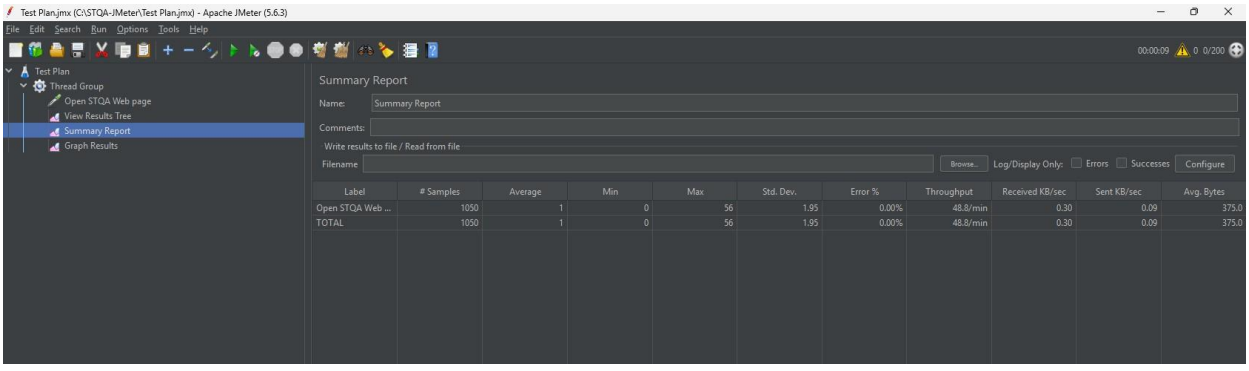
---

14. Result

The load testing of the web application was successfully performed using Apache JMeter. Different load conditions were created using Thread Group, and response time, throughput, and error percentage were observed using JMeter reports and graphs.

Output:





Name : Palak Singh Roll no : 202403123 Class : TYCS-B

